

University of Waterloo
Faculty of Engineering
Department of Electrical and Computer Engineering

ALIVE: Adaptive Live Interactive Virtual Engine

Detailed Design and Project Timeline

Group 2026.04

Prepared by

Harshul Gupta (20998978), h33gupta@uwaterloo.ca
Richa Srivastava (20990875), r4srivas@uwaterloo.ca
Maham Ali (20942119), m327ali@uwaterloo.ca
Jessica Yuan (21007521), j12yuan@uwaterloo.ca
Joey Suh (21011104), jc2suh@uwaterloo.ca
Mairah Hashmi (21011365), mhashmi@uwaterloo.ca

Consultant: Patrick Ziqiang Huang

12 July 2026

Table of Contents

1. High-Level Description of Project	3
1.1 Motivation	3
1.2 Project Objective	3
1.3 Block Diagram	3
1.3.1 Description of System	5
1.3.2 Designed and Not Designed Components	5
2. Project Specifications	7
2.1 Functional Specifications	7
2.2 Non-Functional Specifications	8
3. Detailed Design	9
3.1 Subsystem Design: Biometric Acquisition Unit	9
3.1.1 Final Design Overview	9
3.1.2 Firmware Architecture and Sampling	10
3.1.3 Heart-Rate Channel - PPG Filter Design	11
3.1.4 Peak Detection and Rate Estimates	14
3.1.5 Motion Channel and Yaw Limitations	15
3.2 Subsystem Design: Wireless Communication Layer	17
3.2.1 Final Design Overview	17
3.2.2 Transport Protocol Selection	17
3.2.3 Payload Format and Bandwidth Analysis	18
3.2.4 Latency Budget and Round-Trip Analysis	19
3.2.5 Quantitative Latency Modelling	19
3.3 Subsystem Design: Adaptive Game Engine	20
3.3.1 Final Design Overview	20
3.3.2 Ingestion Layer: Communication Plugin (MQTT/Serial)	21
3.3.3 Data Processing: Biometric State Management	23
3.3.4 Storage: Calibration Profile Store	24
3.3.5 Application: Difficulty Modulation and Game Mechanics	24
3.3.6 Visualization: Player HUD + Telemetry	26
3.3.7 Feedback: Haptic Event Dispatcher	26
4. Discussion and Project Timeline	27
4.1 Evaluation of Final Design	27
4.2 Use of Advanced Knowledge	27
4.3 Creativity, Novelty, Elegance	28
4.4 Student Hours	28
4.5 Potential Safety Hazards	29
4.6 Project Timeline	29
References	30

1. High-Level Description of Project

1.1 Motivation

Most video games respond only to deliberate inputs such as keyboard presses, button clicks, and joystick movements. These capture what a player does but reveal nothing about their physical or emotional state. This limits immersion and makes gameplay feel static, especially in horror or survival games where the experience stays the same regardless of how engaged the player is. In a user research study, biometric methods revealed up to 63% more issues related to players' feelings and immersion than observation alone [3]. Biometric gaming also remains elusive on the developer side with adaptive, biofeedback-driven experiences requiring close cooperation between hardware manufacturers and game developers. With few consumer-level physiological sensors available to build on, studios need to design custom hardware and sensor pipelines from scratch [4].

1.2 Project Objective

ALIVE (Adaptive Live Interactive Virtual Engine) is a biometric-driven gaming system that aims to address these gaps. The system reads physiological signals from the player through a wearable controller, processes them on an embedded microcontroller, and transmits the conditioned data to a game built in Unreal Engine 5 (UE5). The game then adapts its difficulty, pacing, and feedback in response to the player's current state.

1.3 Block Diagram

Figure 1 shows the overall system architecture of ALIVE. The system is divided into three primary subsystems: the Biometric Acquisition Unit (BAU), the Wireless Communication Layer (WCL), and the Adaptive Game Engine (AGE).

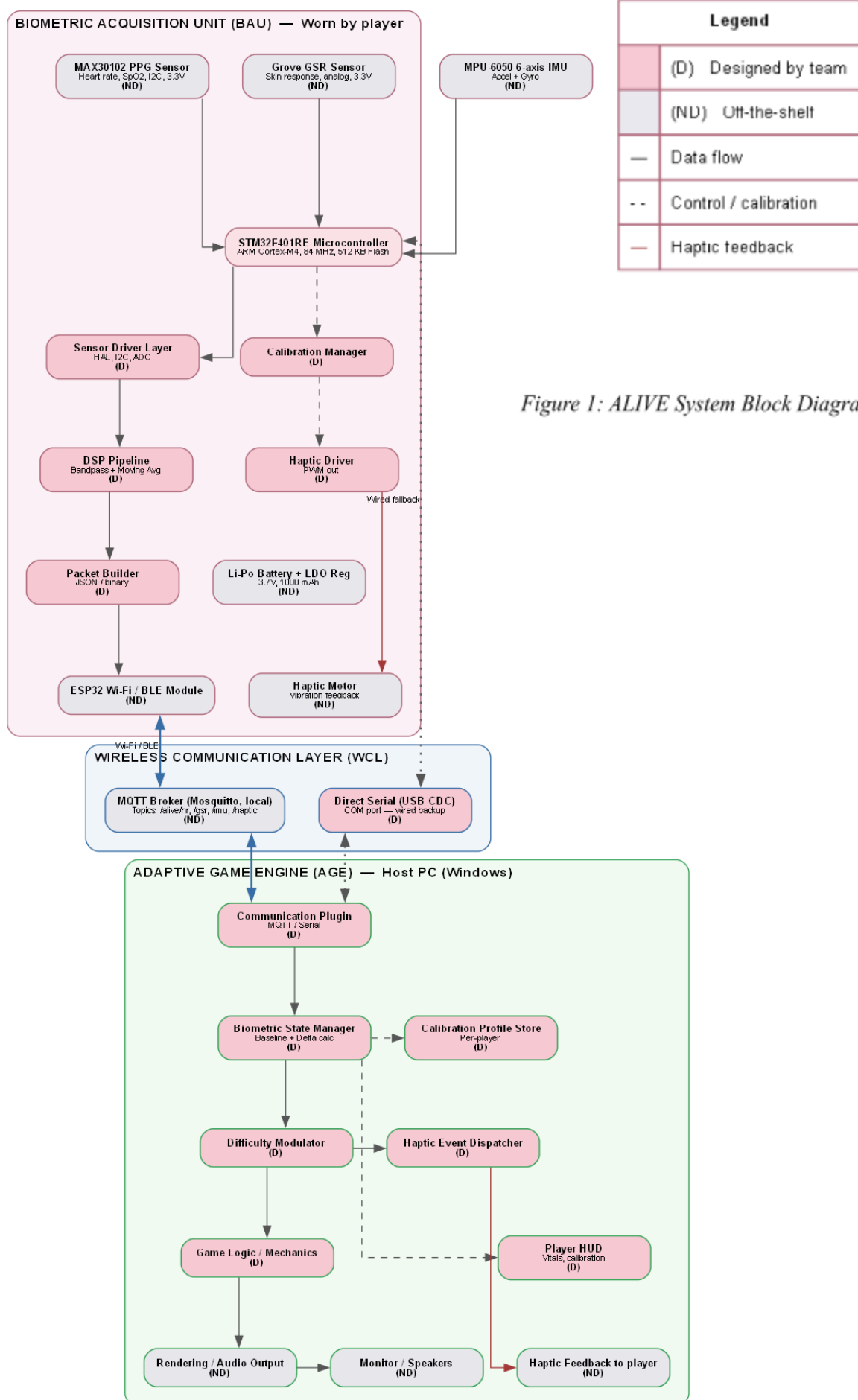


Figure 1: ALIVE System Block Diagram

1.3.1 Description of System

The ALIVE system, shown in the block diagram of Figure 1, is organized into three subsystems: the Biometric Acquisition Unit (BAU), the Wireless Communication Layer (WCL), and the Adaptive Game Engine (AGE). Together, they form a closed loop in which the player's physiological state drives the game, and the game drives feedback back to the player. At the front end, the BAU is a wearable controller that gathers three physiological channels from the player: heart rate from a photoplethysmography (PPG) module, skin conductance from a galvanic skin response (GSR) front end, and motion from a 6-axis inertial measurement unit (IMU). The sensor modules communicate with an STM32F401 microcontroller over an Inter-Integrated Circuit (I²C) interface and the internal analog-to-digital converter. Firmware on the microcontroller samples each channel at a fixed rate, applies per-channel signal conditioning (digital filtering for the PPG channel and oversampled averaging for the GSR channel), and packs the conditioned values into fixed-rate frames. The WCL transports these frames from the BAU to the host running the game. Because the STM32F401 has no integrated radio, the BAU reaches the network through a companion Wi-Fi interface, and biometric frames are published using the Message Queuing Telemetry Transport (MQTT) publish/subscribe protocol to a broker on the host. A wired USB Communications Device Class (CDC) serial link is retained as a deterministic fallback path. The same layer carries commands in the reverse direction, notably haptic events generated by the game. At the back end, the AGE is a game built in Unreal Engine 5 (UE5). It subscribes to the biometric frames, computes each channel as a delta from a per-session calibrated baseline, and feeds those deltas into game systems that adapt difficulty, pacing, and audio in real time. When in-game events occur, the AGE publishes haptic commands back through the WCL to the actuator on the BAU, closing the loop between the player's state and the game's response.

1.3.2 Designed and Not Designed Components

The following is an outline of the components in Figure 1 and the requirements associated for each Designed (D) and Not Designed (ND) component.

- **Biometric sensors - ND.** The PPG, IMU, and GSR sensing elements are off-the-shelf modules selected for the project; designing sensor silicon is outside the project scope. What is designed is everything done with their output (see firmware below).
- **Acquisition and conditioning firmware - D.** The core designed element of the BAU is the firmware: the fixed-rate sampling scheduler, the digital band-pass filter for the PPG channel, the gyro/accelerometer fusion for orientation, and the oversampled averaging for the GSR channel. This is detailed and analysed in Section 3.1.
- **STM32F401 microcontroller - ND hardware, D software.** The microcontroller itself is an off-the-shelf part (ND); the digital control and signal-processing software running on it is designed (D).
- **Haptic driver and actuator - ND hardware, D logic.** The haptic driver IC and linear resonant actuator are off-the-shelf components (ND). The logic that decides when and how to fire haptic events in response to game commands is designed (D).
- **Power subsystem - ND.** The Li-Po cell and its charge/protection circuitry are off-the-shelf parts integrated into the wearable; the team is not designing a custom power board. The safety analysis

(Section 4.5) treats the correct selection and integration of these parts, and the electrical-isolation requirements (N4), as design considerations even though the parts themselves are not designed.

- **Wi-Fi interface/radio module - ND.** The radio is an off-the-shelf module, since the STM32F401 has no integrated Wi-Fi. The specific module and the bridge architecture between it and the STM32 are an open design decision discussed in Section 3.2.
- **MQTT transport - D.** While the broker and protocol standard are off-the-shelf, the design of the message format, topic hierarchy, per-topic quality-of-service settings, and the wired fallback behaviour is a designed element of the WCL.
- **MQTT broker - ND.** A standard broker implementation is configured and run on the host; the team does not implement a broker.
- **Adaptive game and logic - D.** The game-side calibration routine, the delta/normalisation model, the mapping from biometric state to gameplay parameters, and the haptic dispatch are all designed elements of the AGE (Section 3.3).
- **Unreal Engine 5 - ND.** Per non-functional requirement N5, the engine is integrated without modifying its source; the engine itself is not a designed component.

2. Project Specifications

2.1 Functional Specifications

Table 1: Functional Specifications.

ID	Specification	Subsystem	Description	Essential/ Nonessential
F1	Heart rate acquisition	BAU	The system must measure heart rate within plus or minus 5 bpm of a reference pulse oximeter across the range of 50 bpm to 180 bpm	Essential
F2	Galvanic skin response acquisition	BAU	Must measure skin conductance changes and report normalized values relative to a calibrated baseline	Essential
F3	Motion acquisition	BAU	Must report orientation (roll, pitch, yaw) within plus or minus 5 degrees (short-term relative) and report linear acceleration up to plus or minus 2g	Essential
F4	Baseline calibration	BAU + AGE	Must perform baseline calibration routine at session start and use deltas for all gameplay thresholds	Essential
F5	Wireless data transmission	WCL	Must stream biometric data from the BAU to the AGE wirelessly within a 5 m line of sight range	Essential
F6	Game state adaptation	AGE	Unreal Engine 5 game must adjust at least three distinct gameplay parameters (difficulty, enemy pacing, audio cue) in response to input	Essential
F7	Haptic feedback	BAU + AGE	Must trigger haptic events on the BAU in response to in-game events	Essential
F8	Signal conditioning	BAU	Firmware must apply bandpass filtering or oversampled averaging to all biometric channels before transmission	Essential

2.2 Non-Functional Specifications

Table 2: Non-Functional Specifications.

ID	Specification	Subsystem	Description	Essential/ Nonessential
N1	End-to-end latency	Overall	Latency from sensor sampling to game state must not exceed 100ms for motion data and 5s for heart rate and GSR data	Essential
N2	Sampling rate	BAU	The IMU must be sampled at a minimum of 100 Hz, PPG sensor at 25 Hz and the GSR sensor at 10 Hz	Essential
N3	Power autonomy	BAU	Must operate for at least 2 hours on a single charge of a 1000 mAh Li-Po battery	Nonessential
N4	Safety	BAU	All sensor circuits in contact with player skin must be electrically isolated and operate below 5V	Essential
N5	Engine compatibility	AGE	Must integrate with UE5 without requiring engine source changes	Nonessential
N6	Baseline calibration duration	BAU + AGE	Baseline calibration routine must be completed in no more than 60 seconds	Essential
N7	Haptic round-trip latency	BAU + AGE	Round-trip delay between an in-game event and the resulting haptic feedback must be less than 150 ms	Essential

3. Detailed Design

3.1 Subsystem Design: Biometric Acquisition Unit

3.1.1 Final Design Overview

The Biometric Acquisition Unit (BAU) is the wearable front end of ALIVE. It acquires three physiological channels, heart rate, galvanic skin response (GSR), and motion. These are all conditioned in firmware, which assembles the values into fixed-rate frames, and hands them off to the Wireless Communication Layer (Section 3.2). It also receives haptic commands and drives a vibration motor for player feedback (F7, Table 1). The unit is built around an STM32F401RE ARM Cortex-M4F microcontroller, selected for both its hardware floating-point unit (FPU) and DSP instruction set, and for its peripheral set (I²C, SPI, hardware timers, and a 12-bit ADC). Figure 3.1.1 shows the hardware interconnect.

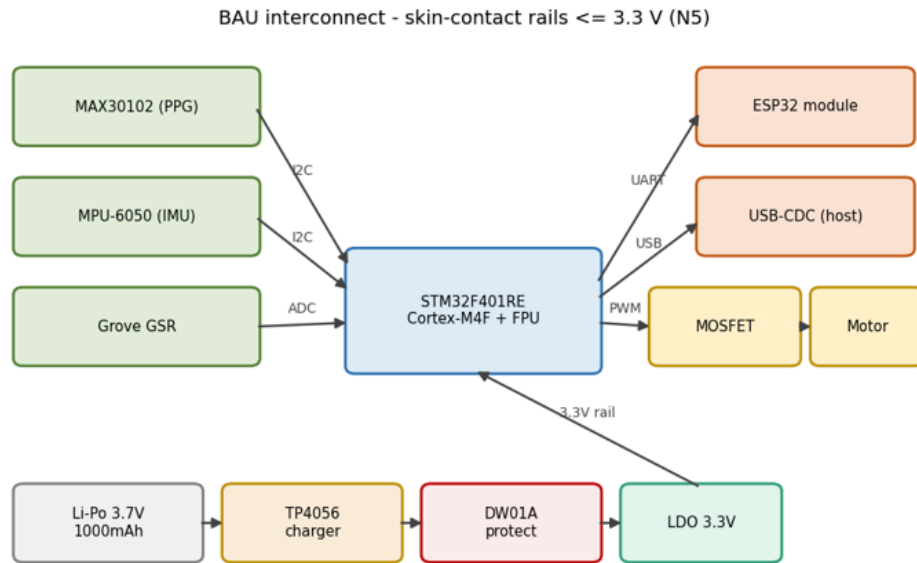


Figure 3.1.1: BAU hardware interconnect. The LDO produces the 3.3 V rail; all skin-contact circuits are held at ≤ 3.3 V (N4).

The sensing elements are the MAX30102 PPG module and the MPU-6050 6-axis IMU, both read over I²C, and the Grove GSR front end, digitized by the STM32 internal ADC. On the microcontroller, firmware performs all of the designed work of the BAU: a fixed-rate sampling scheduler, per-channel signal conditioning (a band-pass filter on the PPG channel and a moving-average filter on the GSR channel, F8), gyro/accelerometer fusion for orientation, a calibration manager that establishes per-session baselines (F4, Table 1), and a packet builder that assembles each conditioned frame (JSON or binary). Player feedback is provided by a PWM-driven vibration motor (F7, Table 1).

Power is supplied by a 3.7 V 1000 mAh Li-Po cell through a protected chain: a charge controller, a dedicated over-charge / over-discharge / short-circuit protection stage, and a low-dropout (LDO) regulator that produces the 3.3 V rail, satisfying the power autonomy requirement N3 (referenced in Table 2). Holding every skin-contact circuit at ≤ 3.3 V is what satisfies the electrical-isolation requirement N4 by design. The protection stage directly addresses the Li-Po hazard identified in the project risk assessment.

3.1.2 Firmware Architecture and Sampling

The firmware runs a bare-metal, timer-driven scheduler rather than a real-time operating system, which is sufficient because the channel set is small, fixed, and periodic. A single hardware timer generates a 1 kHz master tick, and each channel is serviced on a decimated subdivision of that tick: the IMU every 10 ticks (100 Hz), the PPG every 40 ticks (25 Hz), and the GSR every 100 ticks (10 Hz). This meets the minimum rates of N2 exactly and bounds sampling jitter to a single master-tick period (≤ 1 ms) on every channel, because each sample is triggered by the deterministic timer interrupt rather than by variable-latency main-loop code. Low-jitter sampling matters most for the PPG channel, where beat-interval timing (Section 3.1.3) depends on consistent sample spacing. The GSR channel is conditioned with a moving-average filter to satisfy F8, since the GSR is a low-bandwidth signal, and the filtered value is normalized against the per-session calibrated baseline to satisfy the F2 requirement (Table 1). Figure 3.1.2 shows the control flow of one service cycle, including the calibration manager and packet builder stages.

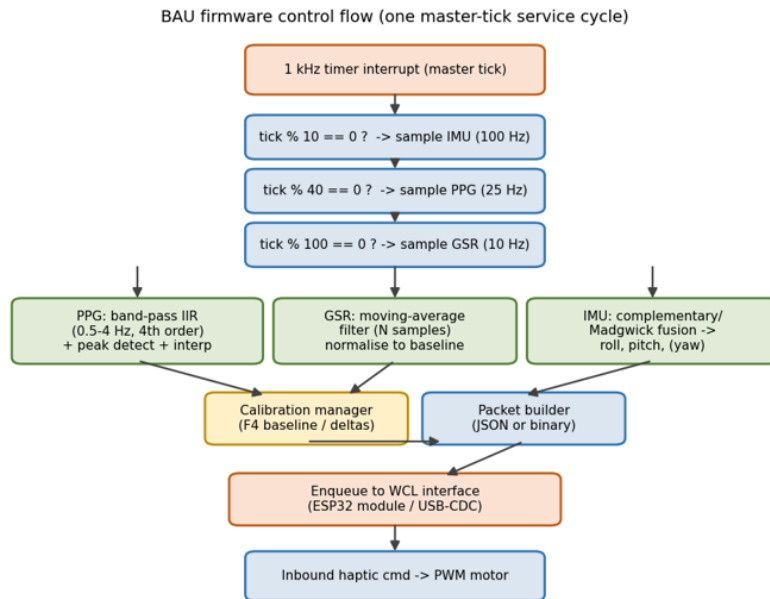


Figure 3.1.2: BAU firmware control flow for one master-tick service cycle.

3.1.3 Heart-Rate Channel - PPG Filter Design

Heart rate is recovered from the photoplethysmography (PPG) waveform produced by the MAX30102, which is sampled at $f_s = 25$ Hz. The acquired signal $x[n]$ is a superposition of three spectrally distinct components:

- a large quasi-DC term from tissue perfusion and ambient light bias;
- low-frequency baseline wander from respiration, vasomotion, and motion of the optical coupling, occupying roughly 0-0.4 Hz;
- the pulsatile arterial component at the cardiac fundamental f_c and its harmonics.

The main design problem is to isolate the pulsatile component so that systolic peaks can be detected and the instantaneous rate computed from peak-to-peak intervals. F1 (Table 1) requires operation over 50-180 bpm. The cardiac fundamental is

$$f_c = \text{bpm} / 60, \text{ so } 50 \text{ bpm} \rightarrow 0.833 \text{ Hz}, 180 \text{ bpm} \rightarrow 3.00 \text{ Hz}.$$

The waveform is also not pure tone, it carries a dicrotic notch and harmonics, so usable energy extends to roughly the second or third harmonic (~6-9 Hz). The Nyquist frequency at 25 Hz is $f_N = f_s / 2 = 12.5$ Hz, which exceeds the 3.0 Hz maximum fundamental and its first harmonic at 6.0 Hz, so both are captured without aliasing. Sampling faster would add negligible waveform fidelity for heart-rate extraction while increasing power and bus traffic, so the N2 minimum of 25 Hz is retained. A band-pass passband of 0.5-4.0 Hz is selected: the lower edge sits below the 0.833 Hz minimum fundamental while rejecting respiration and baseline wander, and the upper edge sits above the 3.0 Hz maximum fundamental while attenuating high-frequency noise.

The filter is designed as an analog Butterworth prototype and mapped to discrete time by the bilinear transform; this is the standard route for a low-order IIR with a maximally-flat passband (no ripple in the cardiac band, which would distort relative peak heights) [1].

Analog prototype and the band-pass transformation

A Butterworth low-pass prototype of order N has a squared magnitude

$$|H(j\Omega)|^2 = 1 / (1 + (\Omega/\Omega_c)^{2N}),$$

i.e. monotonic, maximally flat at $\Omega = 0$, and rolling off at $20N$ dB/decade in the stop-band. The band-pass is obtained from the prototype by the standard low-pass-to-band-pass frequency transformation

$$s \rightarrow (s^2 + \omega_0^2) / (B \cdot s),$$

where the geometric center frequency and bandwidth of the target passband ($\omega_1 = 2\pi \cdot 0.5$, $\omega_2 = 2\pi \cdot 4.0$ rad/s) are:

$$\begin{aligned} \omega_0 &= \sqrt{\omega_1 \cdot \omega_2} = \sqrt{3.1416 \cdot 25.1327} = 8.886 \text{ rad/s} \rightarrow f_0 = 1.414 \text{ Hz}, \\ B &= \omega_2 - \omega_1 = 21.99 \text{ rad/s}, \\ Q &= \omega_0 / B = 0.404. \end{aligned}$$

The bandpass transformation maps each pole to two poles, meaning a bandpass built from the N-th order low pass prototype has order $2N$ and a low-side skirt of $20(2N)$ dB/decade. All orders quoted are band pass orders i.e: 4th order band pass selected below is realized from a 2nd order prototype with 4 poles, and the 2nd order comparison filter is based on a 1st order 2 pole prototype.

Table 3.1.1 reports the attenuation each candidate Butterworth band-pass provides at the principal disturbance frequencies and its group delay across the cardiac band (computed from the realised discrete-time transfer function at $f_s = 25$ Hz; because the bilinear transform warps the frequency axis, attenuation near Nyquist; notably the 10 Hz row differs from an analog-decade extrapolation and should not be re-derived that way). The order is chosen by comparing what each candidate buys (stop-band rejection) against what it costs (group delay). Evaluating $|H(e^{j\omega})|$ at the principal disturbance frequencies:

Disturbance/metric	2nd Order	4th Order
Baseline wander (0.1 Hz)	-15 dB	-30 dB
Respiration (0.25 Hz)	-8 dB	-14 dB
Cardiac band (passband)	0 dB	0 dB
Out-of-band motion (6 Hz)	-7 dB	-11 dB
Ambient flicker (10 Hz)	-16 dB	-32 dB
Group delay, cardiac band	~41-142 ms	~98-283 ms

Table 3.1.1: PPG band-pass filter order comparison (Butterworth, $f_s = 25$ Hz, passband 0.5–4 Hz).

The 4th-order roughly doubles the rejection (in dB) at every disturbance, consistent with the $20N$ dB/decade roll-off doubling from $N = 1$ to $N = 2$ in the simulator. The decisive figure is the 0.1 Hz baseline-wander rejection -30 dB versus -15 dB. Baseline wander is the dominant artifact corrupting peak detection because it is large in amplitude and spectrally adjacent to the lower passband edge, so the extra ~15 dB and a further factor of ~5.6 in amplitude, materially improves peak-detection reliability. Figure 3.1.3 shows the magnitude and group delay response:

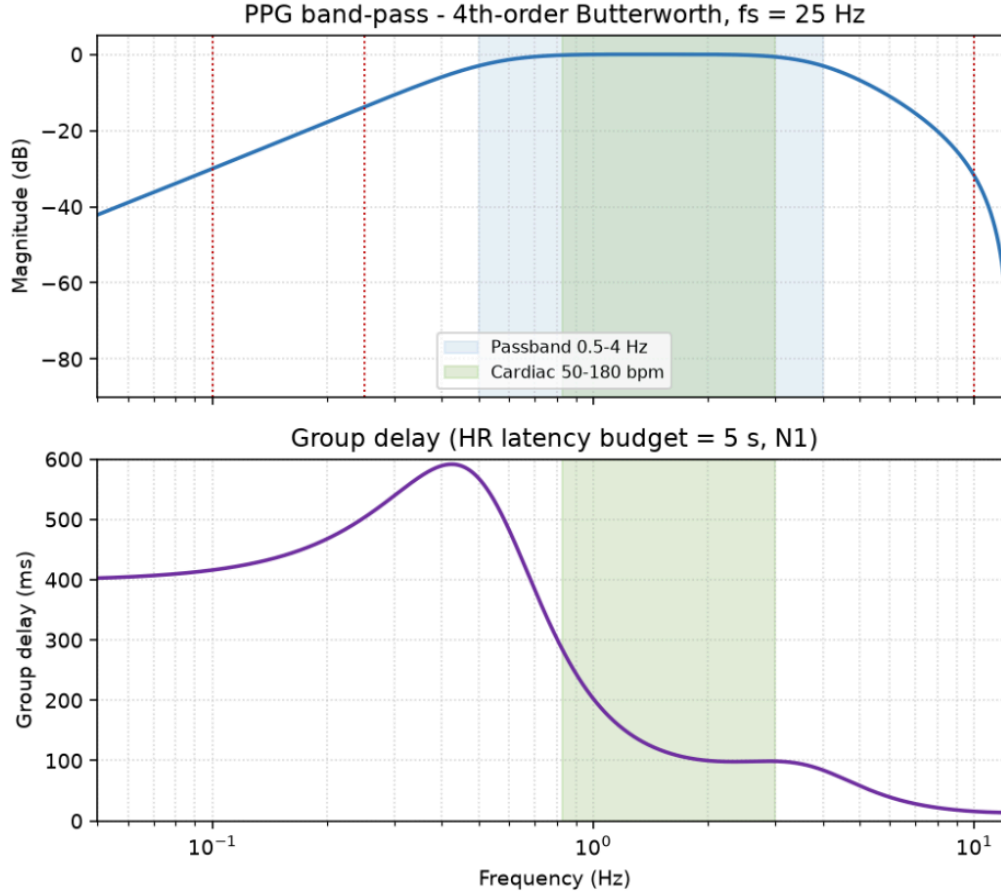


Figure 3.1.3: Magnitude and group delay of the selected 4th-order Butterworth band-pass.

The cost is group delay. The 4th-order filter roughly doubles absolute delay and also doubles the variation of that delay across the cardiac band. Absolute delay is not a problem: heart rate is computed from the interval between successive peaks, so a constant delay shifts both peaks equally and their difference is unchanged. The risk is delay variation: if the rate changes between beats, those beats sit at different fundamentals and are delayed by different amounts, meaning measured intervals are not pure.

Beat-to-beat HR change	$\Delta\tau_g$	Induced interval error	Induced HR error
70 $\rightarrow$ 75 bpm	14.5 ms	14.5 ms	1.33 bpm
120 $\rightarrow$ 126 bpm	1.0 ms	1.0 ms	0.26 bpm
150 $\rightarrow$ 158 bpm	0.3 ms	0.3 ms	0.13 bpm

Table 3.1.2: Interval Error Induced by group delay under changing heart rates

Even at the worst case, 5 bpm jumps at the low end of the range where τ_g varies most steeply, only contributing 1.33 bpm of error.

3.1.4 Peak Detection and Rate Estimates

Filtering only isolates the detected pulse but does not return a rate. We can obtain the rate by locating systolic peaks in the filtered waveform and inverting its peak-to-peak interval. The sampling resolution problem tells us heart rate can be recovered by a beat interval T :

$$HR = 60\,000/T, \text{ where } T \text{ is an interval in ms and } 60\,000 \text{ is one minute in ms}$$

Differentiating with respect to T gives sensitivity of the reported rate:

$$\frac{dHR}{dT} = \frac{-60\,000}{T^2}$$

So an interval error with ΔT produces a rate error:

$$\Delta HR = \frac{60\,000}{T^2} \Delta T$$

Solving for ΔT gets us a beat-interval tolerance according to ± 5 bpm:

Heart Rate	T	ΔT tolerance
60 bpm	1000 ms	± 83.3 ms
120 bpm	500 ms	± 20.8 ms
180 bpm	333 ms	± 9.3 ms

Table 3.1.3: Beat Interval Tolerance corresponding to ± 5 bpm

At 180 bpm, if timing is off by more than 9.3ms our reading will fail F1. From 3.1.3, if $f_s = 25$ Hz and the sample period is 40ms, picking the largest sample quantises peak time to ± 20 ms which is over twice the tolerance. Therefore, external averaging techniques are introduced to meet F1:

1. Parabolic interpolation to fit a curve around a peak sample and its two neighbours to locate peaks between samples [5].
2. Rolling beat average by taking rate over the span of M beats: $HR = \frac{60\,000M}{S}$

The simulation below assumes a 30 dB PPG SNR, typical of good optical contact [5].

	$M = 1$	$M = 2$	$M = 4$
60 bpm	3.03 bpm	1.52 bpm	0.76 bpm
120 bpm	6.21 bpm	3.10 bpm	1.55 bpm
180 bpm	15.68 bpm	7.84 bpm	3.92 bpm

Table 3.1.4: 2σ heart-rate estimation error (bpm) versus averaging window M , at an assumed 30 dB PPG SNR. Assumes parabolic peak interpolation

Therefore, $M = 4$ is the smallest window meeting our F1 requirement across the full range of heart beats. Two dominant error terms are group variation at low bpm and noise jitter at high bpm. If we sum per rate we obtain:

Heart rate	$2\sigma_{\text{noise}} - M = 4$	$\varepsilon_{\text{delay}} (T3.1.2)$	Total	F1 Margin
60 bpm	0.76 bpm	1.33 bpm	2.09 bpm	2.91 bpm
120 bpm	1.55 bpm	0.26 bpm	1.81 bpm	3.19 bpm
180 bpm	3.92 bpm	0.13 bpm	4.05 bpm	0.95 bpm

Table 3.1.5: Combined Heart Rate Error Margins

Parabolic interpolation reduces effective peak-time resolution from ± 20 ms to approximately ± 3 ms [5], which is inside the ± 9.3 ms tolerance of Table 3.1.3 at 180 bpm; quantisation is therefore not a binding term and is already reflected in Table 3.1.4.

3.1.5 Motion Channel and Yaw Limitations

The motion channel uses the MPU-6050, a 6-axis inertial measurement unit integrating a 3-axis MEMS gyroscope and a 3-axis MEMS accelerometer, each digitized by a dedicated 16-bit ADC [2]. Both sensors are read over I²C and sampled at 100 Hz (N2). This subsection specifies how each orientation component and the linear acceleration are obtained, and - using the device's own datasheet figures - quantifies why roll and pitch meet the $\pm 5^\circ$ accuracy of F3 (Table 1) while yaw can only be supported as a short-term-relative quantity.

Configuration and direct measurements

The accelerometer full-scale range is set to $\pm 2g$ (AFS_SEL = 0), giving a sensitivity scale factor of 16,384 LSB/g and a quantization step of about 0.06 mg/LSB [2]. The gyroscope full-scale range is set to $\pm 250^\circ/s$ (FS_SEL = 0), the most sensitive setting, giving 131 LSB/($^\circ/s$); this range comfortably covers the angular rates of hand and wrist motion while maximizing angular resolution. Both sensors' programmable digital low-pass filters are enabled to band-limit the analog signal ahead of the 100 Hz sampling, preventing aliasing of high-frequency mechanical noise.

Roll and pitch: gravity-referenced and bounded

The accelerometer in the IMU measures gravity as a reference. Suppose we denote gravity as g , this g factor provides an absolute, non-drifting reference for the direction of "down." Expressing the measured acceleration vector in the sensor frame as (a_x, a_y, a_z) , the tilt angles follow directly from trigonometry:

$$\begin{aligned} \text{roll } \phi &= \text{atan2}(a_y, a_z), \\ \text{pitch } \theta &= \text{atan2}(-a_x, \sqrt{a_y^2 + a_z^2}) \end{aligned}$$

These references are computed from an instantaneous gravity rather than over time, meaning they do not accumulate error over time nor carry through an error reading. The accelerometer's zero-g initial tolerance

is ± 50 mg on the X/Y axes [2], and a 50 mg offset on a nominal 1 g (1000 mg) reference corresponds to a worst-case static tilt error of

$$\arcsin(0.050 / 1.000) \approx 2.9^\circ,$$

which is within the $\pm 5^\circ$ budget of F3 (Table 1) even before fusion, and is further reduced by the zero-offset removed during calibration.

Yaw: unobservable in a 6-axis IMU

One clear distinction that needs to be made is that yaw about the vertical axis has no absolute reference available to a 6-axis IMU. Rotation about the gravity axis does not change the measured gravity vector, so the accelerometer provides no yaw information, and the MPU-6050 has no magnetometer to supply heading. Rather than adding a magnetometer, the design is scoped to the regime where it is valid: it is treated as a best-effort, short-term-relative quantity, accurate to within $\pm 5^\circ$ only over windows of tens of seconds following a bias re-estimate. This means the AGE will consume yaw only as a relative turn/gesture signal.

The magnitude of this drift can be quantified directly from the datasheet. The gyroscope's initial zero-rate output (ZRO) tolerance is $\pm 20^\circ/\text{s}$ [2], so integrating the raw, uncalibrated bias would produce $20^\circ/\text{s} \times 60\text{ s} = 1200^\circ$ of error in one minute - making a stationary bias-estimation step at session start (part of the F4 calibration routine, Table 1) mandatory rather than optional. Figure 3.1.5 plots the accumulated yaw error against time for a range of representative residual biases, alongside the $\pm 5^\circ$ accuracy limit of F3 (Table 1).

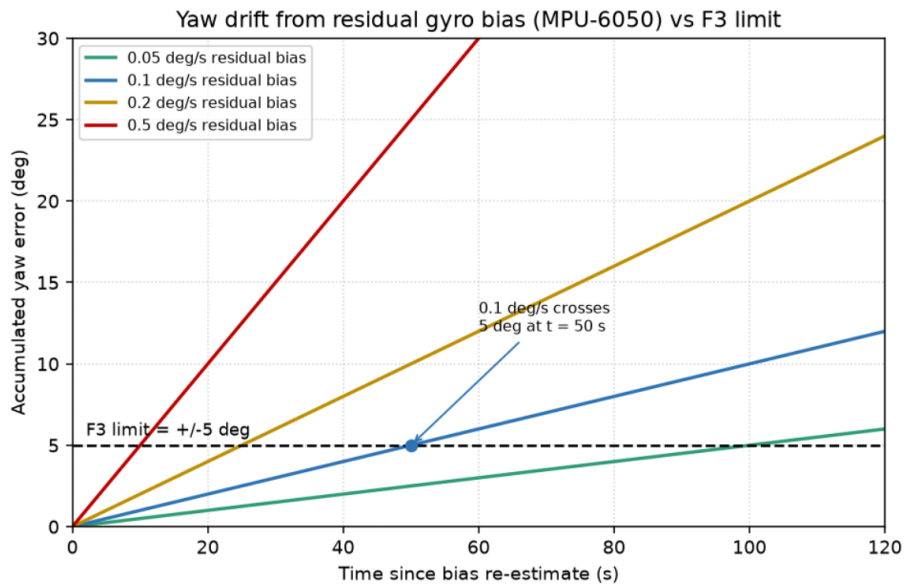


Figure 3.1.5: Accumulated Yaw error from residual gyroscope bias. At a representative residual of $0.1^\circ/\text{s}$, the error reaches 5° after only 50 s. Making Yaw an unsuitable rolling metric.

3.2 Subsystem Design: Wireless Communication Layer

3.2.1 Final Design Overview

The Wireless Communication Layer (WCL) is the transport subsystem of ALIVE. It moves conditioned biometric frames from the Biometric Acquisition Unit (BAU, Section 3.1) to the Adaptive Game Engine (AGE, Section 3.3) and carries haptic commands in the reverse direction, closing the loop between the player's state and the game's response. Because the STM32F401 has no integrated radio, the BAU reaches the network through a companion ESP32 Wi-Fi module, connected to the STM32 over a UART bridge. Biometric frames are published using the MQTT publish/subscribe protocol to a Mosquitto broker running on the host PC, and a wired USB Communications Device Class (CDC) serial link is retained as a deterministic fallback path.

The WCL is the single designed transport in the system: the message format, topic hierarchy, per-topic quality-of-service (QoS) settings, and the wired-fallback behaviour are all designed elements, while the broker, the MQTT standard, and the radio module itself are off-the-shelf (Section 1.3.2). The subsystem must satisfy the wireless-range requirement (F5, 5 m line of sight, Table 1), the haptic round-trip requirement (F7, Table 1) and its 150 ms latency ceiling (N7), and the end-to-end latency budget (N1, ≤ 100 ms motion, ≤ 1 s HR/GSR). Figure 3.2.1 shows the WCL data path: STM32 $\rightarrow$ UART $\rightarrow$ ESP32 $\rightarrow$ Wi-Fi $\rightarrow$ Mosquitto broker $\rightarrow$ UE5 subscriber, with the reverse haptic path following the same route in the opposite direction.

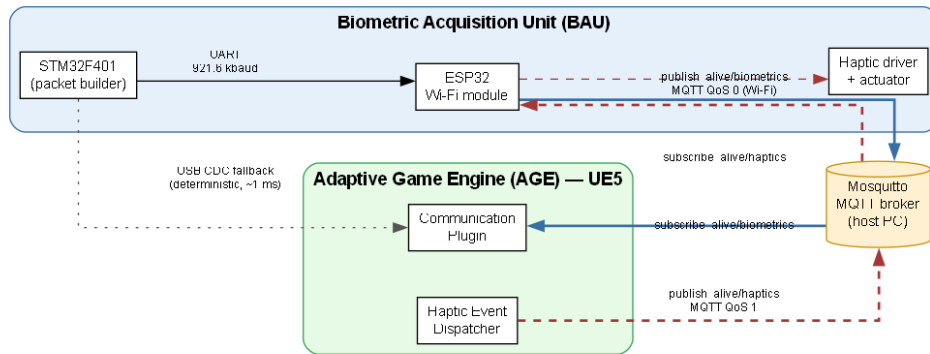


Figure 3.2.1: WCL data path. Biometric frames flow BAU $\rightarrow$ broker $\rightarrow$ AGE; haptic commands flow AGE $\rightarrow$ broker $\rightarrow$ BAU. The wired USB CDC link is a deterministic fallback, bypassing the radio.

3.2.2 Transport Protocol Selection

The central design decision for the WCL is the application-layer transport. Three candidates were evaluated: raw UDP, raw TCP, and MQTT over TCP.

- Raw UDP offers the lowest overhead and latency but provides no delivery guarantees, no automatic reconnection, and no structured topic routing. Lost frames would silently corrupt the AGE's delta calculation.
- Raw TCP guarantees ordered, reliable delivery but requires the team to design its own framing, reconnection, and multiplexing logic on top of the stream, duplicating work the broker already solves.
- MQTT over TCP layers a lightweight publish/subscribe model on TCP. It provides tunable per-topic QoS, automatic reconnection, and a clean topic hierarchy that naturally separates the biometric stream from the haptic command channel.

MQTT was selected because it decouples the BAU and AGE, only needing the broker's address and because its per-topic, QoS lets the two traffic classes be tuned independently. The design assigns:

- **alive/biometrics** → QoS 0 (at most once). The biometric stream is periodic and self-refreshing at 100 Hz; a single dropped motion frame is overwritten 10 ms later and is invisible after filtering. Paying for delivery acknowledgement here would add a round trip of latency for data that is immediately superseded.
- **alive/haptics** → QoS 1 (at least once). A haptic event is a one-off discrete command; dropping it means a missed jumpscare buzz. QoS 1 guarantees delivery, and the AGE-side dispatcher (Section 3.3.7) already de-duplicates by keeping only the highest-priority pending event, so the "at least once" duplicate risk is handled.

These two, fast for the high-rate stream, reliable for the sparse commands is the core designed behaviour of the WCL.

3.2.3 Payload Format and Bandwidth Analysis

Each biometric frame carries the conditioned channel values. Two encodings were considered: human-readable JSON and a packed binary struct. A representative JSON frame:

```
{"hr":72,"gsr":0.34,"roll":12.5,"pitch":-3.2,"yaw":88.1}
```

is approximately 56 bytes of payload. The equivalent packed binary frame — one uint8 for HR, one uint16 fixed-point for GSR, and three int16 for orientation, is 9 bytes of payload. The motion channel sets the frame rate at 100 Hz (N2). The sustained application-layer throughput for each encoding is

$$R_{JSON} = 56 \text{ B} \times 100 \text{ Hz} = 5,600 \text{ B/s} \approx 44.8 \text{ kbit/s}$$

$$R_{BIN} = 9 \text{ B} \times 100 \text{ Hz} = 900 \text{ B/s} \approx 7.2 \text{ kbit/s}$$

Adding fixed per-message overhead — roughly 2 bytes MQTT fixed header + ~14 bytes for the alive/biometrics topic string + ~40 bytes TCP/IP header per packet, the binary encoding's total on-wire rate is on the order of 65 kbit/s, still under 0.33% of the ~20 Mbit/s practical throughput of a 2.4 GHz 802.11n link at 5 m. We observed that bandwidth is not the binding constraint; latency and per-packet overhead are. Binary was selected for the production stream because it minimizes serialization time on the

STM32 and the number of TCP segments per second, while JSON is retained as a debug-mode encoding because the AGE's telemetry widget (Section 3.3.6) is far easier to develop against a readable stream.

3.2.4 Latency Budget and Round-Trip Analysis

The WCL must fit inside two budgets: the N1 end-to-end sensor-to-game latency (≤ 100 ms for motion) and the N7 haptic round-trip (< 150 ms). The WCL contributes the transport portion of both. The one-way transport latency decomposes as:

$$T_{\text{transport}} = T_{\text{serialize}} + T_{\text{UART}} + T_{\text{radio}} + T_{\text{broker}} + T_{\text{deserialize}}$$

The UART bridge between the STM32 and ESP32 runs at 921 600 baud. A 9-byte binary frame plus framing (~ 11 bytes) transmits in:

$$T_{\text{UART}} = \frac{11B \times 10 \text{ bits/B}}{921600 \text{ bits/s}} = 0.12 \text{ ms}$$

which is negligible against the dominant and most variable term (Wi-Fi + broker hop), $T_{\text{radio}} + T_{\text{broker}}$, which measurement and vendor data place in the 20–40 ms range under typical conditions, but which can spike under channel contention. The haptic round-trip in Section 3.3.7 budgets 20–40 ms typical / up to ~ 120 –140 ms worst case for exactly this reason, and this analysis confirms the WCL is the dominant contributor to that figure. This is the engineering justification for retaining the wired USB CDC fallback: the fallback removes $T_{\text{radio}} + T_{\text{broker}}$ and replaces it with a deterministic ~ 1 ms USB bulk transfer, collapsing worst-case tail and guaranteeing a latency-bounded path whenever a wire is available.

3.2.5 Quantitative Latency Modelling

To justify the QoS split and the fallback decision, the round-trip latency was modelled for three configurations across increasing channel load. Under light load, the Wi-Fi hop behaves near its floor; as offered load rises, contention on the shared 2.4 GHz medium introduces queueing delay that grows non-linearly, following the classic $T = T_0 / (1 - \rho)$ form of an M/M/1 queue, where ρ is channel utilization. The wired fallback with a dedicated link stays flat.

The graph below plots modelled round-trip latency versus channel utilization for (a) MQTT QoS 1 over Wi-Fi, (b) MQTT QoS 0 over Wi-Fi, and (c) the wired USB CDC fallback, against the N7 150 ms ceiling.

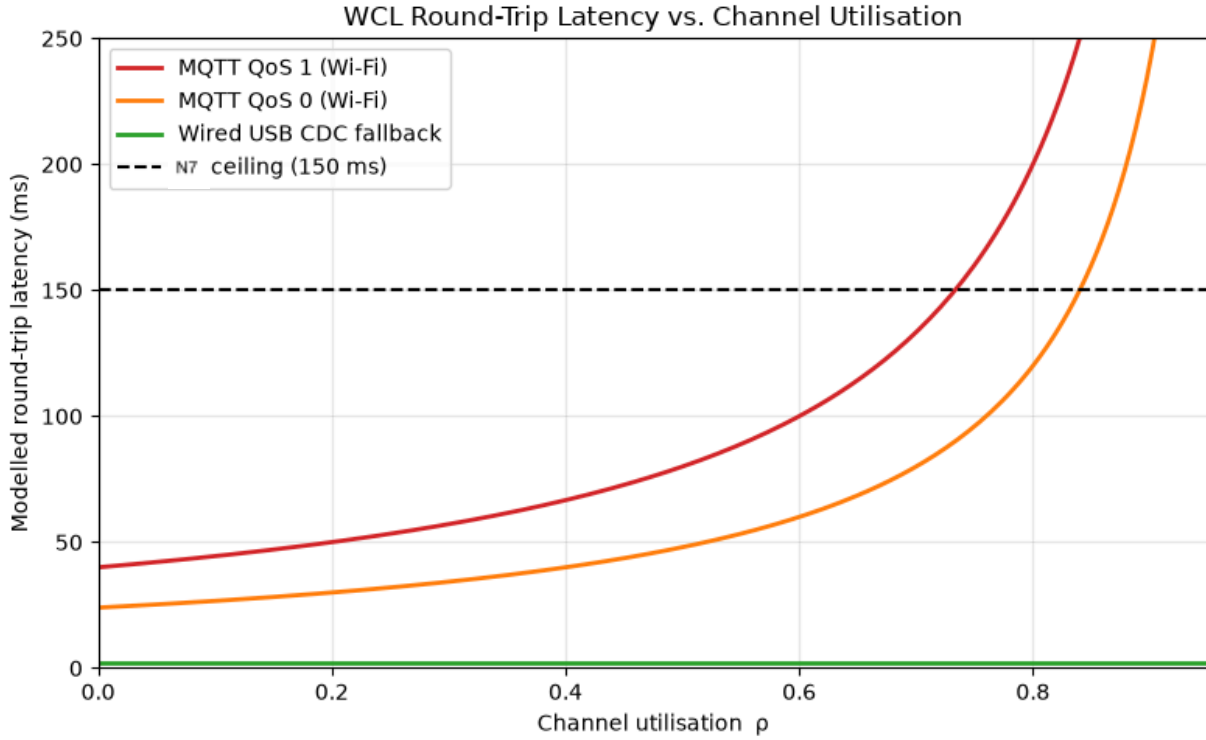


Figure 3.2.2: Modelled WCL round-trip latency vs. channel utilization. Wi-Fi paths degrade sharply as the shared medium saturates; QoS 1 crosses the N7 ceiling earlier than QoS 0 due to its acknowledgement round-trip. The wired fallback stays flat and deterministic.

3.3 Subsystem Design: Adaptive Game Engine

3.3.1 Final Design Overview

The Adaptive Game Engine (AGE) subsystem is hosted within Unreal Engine 5 (UE5). To minimize architectural overhead, the subsystem’s data pipeline is organized into 6 components. Each component is outlined within the AGE block in Figure 3.3.1. Data flows dynamically across a forward processing pipeline and a low-latency feedback loop.

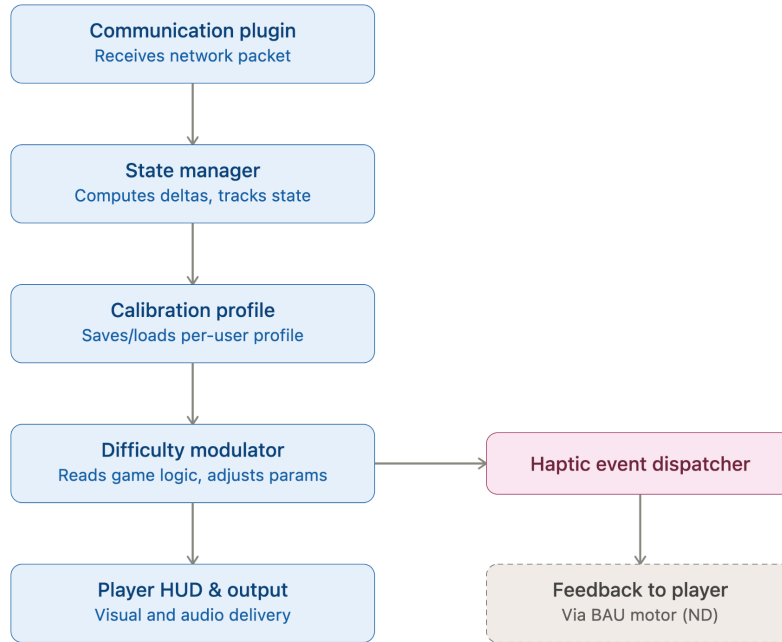


Figure 3.3.1: AGE Subsystem Block Diagram, Blue = AGE forward pipeline, Pink = haptic dispatcher

The forward pipeline begins at the Ingestion Layer, where a dedicated Communication Plugin receives incoming network packets via MQTT or serial connections. Raw biophysical data is passed to the Biometric State Manager to filter noise and calculate baseline deltas, tracking instantaneous physiological changes. These states flow into the Calibration Profile Store, which saves and loads user-specific baselines to adapt the system to individual players.

The integrated Difficulty Modulation and Game Mechanics layer reads these calibrated biometric states alongside active game logic to alter gameplay parameters in real time. For example, it can dynamically degrade tool stability in a surgical simulation or scale enemy aggression and tracking in a horror/monster-chasing scenario. Concurrently, the pipeline pushes updated metrics and systemic shifts to the Player HUD and Telemetry Visualization layer for real-time visual and auditory feedback.

Complementing this flow is a responsive feedback loop. When structural thresholds or critical events are breached within the UE5 game logic (e.g., a sudden spike in player stress), the engine triggers the Haptic Event Dispatcher. This component translates abstract gameplay events into structured haptic commands and routes them back to the hardware layer, delivering precise tactile feedback and closing the interaction loop.

3.3.2 Ingestion Layer: Communication Plugin (MQTT/Serial)

The communication plugin serves as the AGE's network gateway, bridging incoming WCL stream frames into the game engine. To ensure low-latency processing, the ingestion layer uses a multithreaded architecture: on session initialization, a dedicated worker thread handles continuous network polling, message parsing, and buffer management.

MQTT over a local Mosquitto broker is the transport mechanism. Incoming packets use a lightweight string-tokenized format, parsed into three sensor domains: `hr_bpm` (PPG heart rate), `gsr_conductance` (normalized GSR), and `imu_roll/pitch/yaw` (Euler angles from 6-axis IMU fusion).

A single-threaded polling model was initially considered, but rejected because stochastic packet arrival times coupled with frame drops caused data loss on the high-frequency IMU stream. The multithreaded design with a thread-safe circular ring buffer decouples ingestion from the render loop, eliminating frame-rate-induced starvation. MQTT was selected for its publish/subscribe topology and ability to meet the wireless range requirement.

Quantitative Technical Analysis

To validate the multi-threaded ingestion pattern and choose an optimal size for the circular queue, an engineering queue model was established. Let the arrival rate of biometric data frames across all combined sensor channels be denoted as λ . Based on the hardware transmission profiles, the incoming aggregate frame arrival velocity running over the MQTT pipeline operates at the highest common denominator, which is: $\lambda = 100 \text{ frames/second}$

The game engine ticks at a variable consumption rate denoted as μ . Assuming a worst-case heavy graphics rendering load where the game engine struggles to maintain a baseline of 5 fps, the main thread's data consumption sequence checks the buffer at: $\mu = 5 \text{ checks/second}$

Because each frame tick on the main thread processes the entire depth of the accumulated buffer elements in a single block read operation, the capacity of the circular array must be large enough to safely contain the frames generated during the engine's processing down-period t_{stall} . At 5 fps, the maximum engine loop step duration is:

$$t_{stall} = \frac{1}{\mu} = \frac{1}{5} \approx 200 \text{ ms}$$

The minimum required buffer size B_{min} to prevent overwrite loss and frame dropping during this stall period is calculated by:

$$B_{min} = \lambda \times t_{stall} = 100 \times 0.200 = 20 \text{ frames}$$

To buffer against anomalous OS thread context-switching delays or prolonged rendering blocks, the safety margin is scaled using a base-2 integer constraint to 256. Allocating a fixed buffer depth of 256 elements provides a safety factor (elements ÷ frames) of approximately 12.8×. This allocation ensures that zero packets are dropped at the input stage, successfully preserving input stability for the downstream baseline calibration modules.

Satisfying Project Specifications

By keeping asynchronous socket polling on a worker thread, the main thread does not block on network I/O. This ensures that processing the 100 Hz motion data easily fulfills the $\leq 100\text{ms}$ motion latency envelope defined in the end-to-end latency requirement (N1).

To satisfy the haptic feedback requirement (F7, Table 1), the plugin handles bidirectional routing. It exposes a synchronous command channel to dispatch downstream vibration intensity mappings back to

the hardware platform, keeping the round-trip feedback latency well below the required 150 ms limit (N7).

3.3.3 Data Processing: Biometric State Management

The Biometric State Manager serves as the analytical core of the Adaptive Game Engine (AGE). It consumes the raw, buffered telemetry streams (HR, GSR, IMU) from the Communication Plugin and transforms them into standardized, actionable metrics. Because human physiological responses vary drastically from person to person, the engine cannot rely on hard-coded sensor values to determine if a player is stressed.

Instead, the State Manager utilizes a statistical dynamic thresholding model. It initiates a localized calibration phase to establish a resting physiological profile for the current user, and then continuously computes standardized deltas (Z-scores) during active gameplay. These relative deltas are dispatched to the downstream game mechanics to drive difficulty modulation.

Mathematical Formulas of Z-Score Deltas

To remove individual baseline variations, the State Manager translates incoming raw physiological parameters into dimensionless statistical deviations. For each incoming frame, individual channel Z-scores are computed continuously using the baseline parameters established during the calibration phase:

$$Z_{HR} = \frac{HR_{raw} - \mu_{HR}}{\sigma_{HR}} \quad Z_{GSR} = \frac{GSR_{raw} - \mu_{GSR}}{\sigma_{GSR}} \quad Z_{Motion} = \frac{Motion_{raw} - \mu_{motion}}{\sigma_{motion}}$$

where HR_{raw} , GSR_{raw} , and $Motion_{raw}$ are the instantaneous values parsed by the Ingestion Layer (3.3.2). μ and σ represent the stored baseline mean and standard deviation for each respective channel. These calculated metrics are output dynamically to the Difficulty Modulation and Game Mechanics layer (Section 3.3.5), where they are synthesized into the $Z_{composite}$ core using targeted weights.

Mechanics of the Baseline Calibration Routine

The 45-second automated profiling routine operates as follows:

1. **Data Ingestion Phase:** Upon trigger during the game's initial menu or loading sequence, the State Manager opens a rolling allocation window. It accumulates samples from the Ingestion Layer over exactly 45 seconds at the specified channel frequencies: 1,125 samples for Heart Rate (25 Hz), 450 samples for GSR (10 Hz), and 4,500 samples for IMU data (100 Hz).
2. **Statistical Profiling:** Once the timer expires, the subsystem calculates the steady-state mean and standard deviation for each sensor array.
3. **Profile Handoff:** These calculated baseline matrices are immediately dispatched to the Calibration Profile Store (Section 3.3.4) for local JSON serialization and caching, ensuring zero overhead on successive session launches.

Satisfying Project Specifications

The subsystem executes an automated profiling routine during the game's initial menu/loading sequence. The routine spans exactly 45 seconds (comfortably below the 60-second maximum requirement) to capture enough samples to establish a statistically significant resting mean and variance. This timing correctly aligns with the baseline calibration requirement (N6).

By strictly passing relative deviation scores rather than absolute biometric integers to the gameplay components, the engine guarantees a normalized gameplay experience. A player with a natural resting heart rate of 60 bpm and a player with a resting heart rate of 90 bpm will trigger the exact same game state adaptations when they experience proportional physiological spikes, therefore satisfying the requirement of using delta-driven thresholds (F4 + F6, Table 1).

3.3.4 Storage: Calibration Profile Store

The Calibration Profile Store serves as the persistence and state-management layer within the Adaptive Game Engine (AGE). This subsystem is implemented as a decoupled, local data access object (DAO) integrated into the Unreal Engine 5 (UE5) subsystem architecture using a serialized JSON file scheme stored locally on the host PC.

Upon the completion of the 45-second baseline calibration routine managed by the Biometric State Manager, the Calibration Profile Store captures the calculated statistical baseline metrics, specifically the steady-state mean and standard deviation for heart rate (PPG), galvanic skin response (GSR), and localized IMU motion thresholds. These profiles are serialized alongside a unique player identifier, a cryptographic checksum for file integrity, and historical scaling factors, allowing the engine to instantly load historical Baselines for returning players or update existing profiles dynamically as gameplay sessions progress and physiological baselines drift.

Satisfying Project Specifications

This design directly satisfies the Baseline Calibration functional specification (F4, Table 1) by providing a structured, deterministic medium to record and load user baselines, ensuring that gameplay thresholds are driven entirely by localized deltas relative to a pre-recorded baseline rather than generic, hardcoded values. Because specification N1 mandates strict end-to-end latencies, under 100 ms for motion data and under 1s for heart rate and f, the Calibration Profile Store is structurally engineered to execute serialization and deserialization operations asynchronously on a dedicated, low-priority worker thread using UE5's `FNonAbandonableTask` framework. By offloading file I/O operations from the main game thread, the subsystem eliminates the risk of CPU frame-time spikes or stuttering during real-time profile lookups or mid-game recalibration saves, thereby ensuring non-functional execution requirements are maintained seamlessly during intense gameplay states.

3.3.5 Application: Difficulty Modulation and Game Mechanics

The Difficulty Modulation and Game Mechanics layer is where the AGE converts standardized biometric deltas into changes the player actually experiences, such as enemy pacing, difficulty level and audio cues.

The three Z-scores provided by the Biometric State Manager (as mentioned in Section 3.3.3) are Z_{HR} , Z_{GSR} , and Z_{motion} , and since gameplay can only act on one difficulty decision at a time, these three separate scores must first be combined into a single number representing overall player engagement. This is achieved with the weighted sum:

$$Z_{composite} = 0.5 * Z_{HR} + 0.3 * Z_{GSR} + 0.2 * Z_{motion}$$

Here, heart rate is weighted the highest at 50% as the cleanest signal, while GSR is weighted at 30% and motion at 20%. This is because the GSR and motion are more easily disturbed by factors unrelated to stress (ex: accidental arm movement).

This design organizes the combined biometric state into four discrete engagement tiers: Calm, Engaged, Stressed and Panic. These are organized as follows:

Table 3.3: Engagement-Tier Table.

Tier	$Z_{composite}$ range	Enemy behaviour	Encounter Interval	Audio
Calm	<0.5	Reduced aggression	45s	Ambient only
Engaged	0.5 - 1.5	Reduced aggression	30s	Ambient + low tension
Stressed	1.5 - 2.5	+25% tracking speed	18s	Tension + stinger cues
Panic	> 2.5	+50% tracking speed	10s (floor)	Full mix, distortion filter

This approach avoids any frame-to-frame jitter that a continuous mapping from sensor noise alone would introduce. We further apply a hysteresis band of ± 0.15 around each threshold, plus a 2-second minimum between transitions, which prevents the tier from flickering back and forth near a boundary. An example of this can be seen with a player at $Z_{composite}$ at 1.5, who would need to reach 1.65 to enter Stressed, and must drop back to 1.35 to return to Engaged.

Satisfying Project Specifications

This satisfies F6 (Table 1), which requires at least three gameplay parameters (difficulty, pacing, audio) to change in real time with biometric input, since each tier transition updates all three from one state variable. The applied 2-second throttle is independent of N1:N1, which governs how fast the biometric data reaches the game's internal state, not how fast that state may change visible gameplay, so the pipeline still meets N1 while gameplay stays stable.

3.3.6 Visualization: Player HUD + Telemetry

This Visualization layer takes the biometric state and shows it to two different audiences: the player, who needs a subtle sense that the game is reacting to them and the dev team, who need to see the raw numbers to test the system. These are built as two separate Unreal Motion Graphics widgets in UE5, both listening to the same broadcast from the Biometric State Manager. The player is shown a minimal cue (ex: a soft pulse synced vignette), while the telemetry widget shows raw HR/GSR/IMU values, the current $Z_{\text{composite}}$, active tier and measured latency.

Biometric data can arrive up to 100 Hz, but redrawing UI elements 100 times a second is wasteful and can compete with the game's rendering for frame time. Instead, both widgets refresh on a fixed 10 Hz timer (or every 100 ms), which is a lot faster than a human can perceive a heart-rate number changing, but light enough not to affect performance.

Satisfying Project Specifications

Here, the player and telemetry views are using separate widgets, meaning the player-facing HUD stays minimal and unobtrusive without sacrificing the detailed view needed to verify F6 and F7 (Table 1) during testing, and the telemetry widget can be stripped out entirely for a shipping build. The UI refresh is capped at 10 Hz, which is independent of the underlying 100Hz data rate, and this keeps the HUD from adding measurable frame time overhead and protects the N1 latency budget for the system that actually needs it. The telemetry widget also displays the live measured haptic round-trip latency (as mentioned in Section 3.3.7) and data ingestion latency, which is used to verify that the N1 and N7 requirements are being met during testing rather than only in theory.

3.3.7 Feedback: Haptic Event Dispatcher

The Haptic Event Dispatcher turns discrete in-game moments such as jumpscare, enemy hits and near-misses into a vibration on the Biometric Acquisition Unit motor. This must be achieved within N7's 150 ms round-trip limit. The game raises one-off events, as opposed to a continuous vibration signal every tick. These are each mapped to a preset vibration pattern that is stored in a lookup table. This keeps each message tiny (just an eventId) so it does not clog the wireless link.

If several events fire close together, the dispatcher keeps only the single highest-priority pending event and drops the lower-priority ones that arrive while it is still in flight, guaranteeing responsiveness and avoiding any disconnect from triggering events.

The 150 ms budget breaks down as follows:

Table 3.3.1: Haptic event latency budget breakdown

Stage	Latency	Worst-case latency
Game decision + event lookup	~1-2 ms	~2-3 ms
WCL transmission (MQTT over Wifi)	~20-40 ms typical	~80-100 ms (retries, contention, backoff on a congested channel)

BAU frame parse + PWM actuation	< 5 ms	~5-8 ms
Actuator rise time	~20-30 ms	~30 ms (cold motor, low battery)
Total, typical case	~50-80 ms	~120-140 ms

NOTE: Considering the worst case, this stays under the 150 ms ceiling. WiFi transmission is the dominant and most variable term, so it is the main driver of the worst-case bound.

Satisfying Project Specifications

This satisfies F7 (Table 1) by triggering haptic events in response to in-game events and satisfies N7 directly, since even the worst-case estimate stays under 150 ms, with WiFi transmission as the most variable and the dominant term. This is also why the wired USB CDC fallback (as mentioned in Section 1.3.1) is kept as an option; switching to it removes the variable WiFi term from the budget entirely, giving a guaranteed latency path whenever a wired connection is available.

4. Discussion and Project Timeline

4.1 Evaluation of Final Design

The final design meets the project objective. Each subsystem traces directly to the specifications in Section 2.

The BAU satisfies acquisition and conditioning. The 4th-order Butterworth band-pass recovers heart rate within F1 (Table 1), all channels sample at the N2 (Table 2) minimum rates with sub-millisecond jitter, and roll and pitch stay inside the $F3 \pm 5^\circ$ budget (Table 1). Yaw is correctly scoped as a short-term relative signal rather than overclaimed.

The WCL meets F5, F7, N7, and the transport share of N1 (Table 1 & Table 2). The QoS-differentiated MQTT design matches each traffic class to its reliability need, and the latency model keeps the haptic round trip under 150 ms across realistic channel loads. The wired fallback guarantees a deterministic path when Wi-Fi contention threatens the budget.

The AGE satisfies F4, F6, F7, N6 and N7 (Table 1 & Table 2) through per-player calibration and a four-tier engagement model driven by a single composite state. Hysteresis and transition throttling keep gameplay stable without violating N1. The main residual risk is Wi-Fi variability under contention, which is why the wired fallback is a first-class path.

4.2 Use of Advanced Knowledge

The design draws on upper-year ECE knowledge across all three subsystems:

Digital signal processing: The PPG channel applies a Butterworth filter design, the bilinear transform, and the low-pass-to-band-pass transformation, with order chosen by a stop-band-rejection versus group-delay trade-off (Section 3.1.3).

Measurement and error analysis: The motion channel propagates datasheet tolerances into orientation error, deriving both the tilt bound and the yaw drift rate (Section 3.1.4).

Networks and queueing theory: The WCL latency model uses the M/M/1 form to predict how the shared medium degrades under load (Section 3.2.5).

Embedded and real-time systems: The BAU uses a bare-metal, timer-driven scheduler with bounded jitter, and the AGE offloads file I/O to a worker thread to protect frame time (Sections 3.1.2, 3.3.4).

Statistics: The AGE applies Z-score normalization and hysteresis to stay robust to inter-player variation (Sections 3.3.3, 3.3.5).

4.3 Creativity, Novelty, Elegance

The most novel aspect is the closed biometric loop: the player's physiology drives the game, and the game drives haptic feedback back to the player, on low-cost, reproducible hardware.

Several choices solve problems by scoping rather than adding hardware. Yaw is re-scoped to a valid short-term relative regime instead of adding a magnetometer. The transport deliberately picks unreliable QoS 0 for the self-refreshing stream and reserves reliable QoS 1 for the sparse commands that matter, buying latency headroom. Delta-driven Z-scores make players with different resting baselines trigger identical adaptations under proportional stress. The wired fallback doubles as the guaranteed-latency escape hatch that collapses the worst-case Wi-Fi tail.

4.4 Student Hours

Table 4.4: Student Hours

Name	Hours
Joey Suh	61
Jessica Yuan	62
Mairah Hashmi	60
Maham Ali	60
Richa S	58
Harshul Gupta	58

4.5 Potential Safety Hazards

Lithium-polymer battery: The Li-Po cell is the primary hazard. Over-charge, over-discharge, puncture, or short circuit can cause thermal runaway. The protection stage and charge controller mitigate this by design. During construction, the cell is never charged unattended, is stored in a fireproof bag, and is inspected for swelling before use.

Skin contact: Every skin-contact circuit stays at 3.3 V or below behind the LDO, satisfying N4. No mains-referenced or unregulated voltage reaches the wearable, and the USB path is isolated as a potential fault route.

Skin irritation: GSR electrodes and gel can irritate skin. Contact surfaces are skin-safe and cleaned between users.

Assembly: Soldering burns, flux fumes, and ESD damage are managed with ventilation, eye protection, and ESD precautions.

Actuator: The vibration motor is mechanically secured so it cannot detach or pinch during wear

4.6 Project Timeline

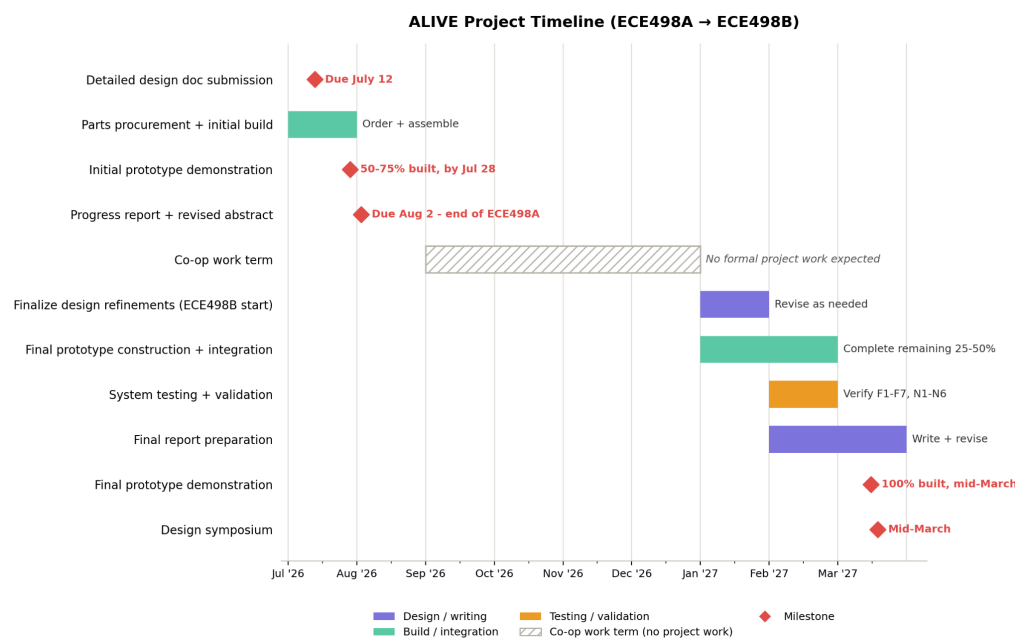


Figure 4.6 ALIVE Project Timeline ECE498A to ECE498B

References

- [1] A. V. Oppenheim and R. W. Schaffer, *Discrete-time signal processing*, 3rd ed. Upper Saddle River: Pearson, 2010.
- [2] "MPU-6000 and MPU-6050 Product Specification Revision 3.4 MPU-6000/MPU-6050 Product Specification," 2013. Available:
<https://d17t6iyxenbwp1.cloudfront.net/s3fs-public/2026-04/ps-mpu-6000a-00-mpu-6000-and-mpu-6050-datasheet.pdf?VersionId=eMVDQ08Qz7kahOvvE6xCs8WtRhxeKFIH>
- [3] P. Mirza-Babaei, S. Long, E. Foley, and G. McAllister, "Understanding the contribution of biometrics to games user research," in Proc. DiGRA 2011 Conf.: Think Design Play, 2011.
- [4] L. E. Nacke, M. Kalyn, C. Lough, and R. L. Mandryk, "Biofeedback game design: Using direct and indirect physiological control to enhance game interaction," in Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '11), Vancouver, BC, Canada, 2011.
- [5] H. J. Baek, J. Shin, G. Jin, and J. Cho, "Reliability of the parabola approximation method in heart rate variability analysis using low-sampling-rate photoplethysmography," *Journal of Medical Systems*, vol. 41, no. 12, p. 189, 2017.